Columbia University
Department of Industrial Engineering and Operations Research

IEOR E4004: Optimization Models and Methods

Project I

Eliminating child care Deserts in New York State through Optimization

Group 7

Xiangxiang Li (UNI: xl3580)
Yuqing Tao (UNI: yt2983)
Thomas Flamand (UNI: tf2636)
Tianci Chen (UNI: tc3507)

Instructor: Dr. Yaren Bilge Kaya

Teaching Assistants: Salam Afiouni & Tongtong Chen & Mohamed Lakhnichi

Submission Date: October 30, 2025

Contents

1 Introduction **2**

2 Background **2**

3 Methods **2**

3.1 The Problem of Budgeting 2

3.1.1 Problem Statement and Assumptions 2

3.1.2 Sets, Parameters, and Data 3

3.1.3 Decision Variables 4

3.1.4 Objective Function 4

3.1.5 Constraints 4

3.1.6 Solution Approach 5

3.2 The Problem of Realistic Capacity Expansion and Location 5

3.2.1 Problem Statement and Assumptions 5

3.2.2 Sets, Parameters, and Data 6

3.2.3 Decision Variables 6

3.2.4 Expansion Cost 6

3.2.5 Location Feasibility Constraint 6

3.2.6 Coverage Constraints 7

3.2.7 Objective 7

4 Results and Conclusions **7**

5 Fairness Considerations (Optional Extension) **9**

6 Future Work: Toward a Combined Objective **10**

7 Appendix (Code) **11**

1 Introduction

This project addresses the issue of **child care deserts** in New York State (NYS) — regions where the demand for licensed child care far exceeds the available capacity. More than half of American children live in such areas, and the NYS government aims to ensure that every region has sufficient access to affordable child care services.

The project is divided into two main parts:

- An **idealistic budgeting model** where new facilities and expansions can occur without spatial or operational limitations.
- A **realistic expansion and location model**, incorporating more accurate cost functions and minimum distance constraints between facilities.

Our goal is to develop optimization models that determine the minimum total funding required for NYS to eliminate child care deserts under both scenarios.

2 Background

A "child care desert" is defined according to state-level criteria:

- In **high-demand** areas (employment $\geq 60\%$ or average income $\leq \$60,000$), an area is considered a desert if the number of available slots is $\leq 50\%$ of the child population aged 0–12.
- In **normal-demand** areas, the threshold is one-third of the population aged 0–12.

Additionally, New York State requires that for children aged 0–5, the available number of child care slots must be at least two-thirds of the corresponding population.

3 Methods

3.1 The Problem of Budgeting

3.1.1 Problem Statement and Assumptions

The goal of the budgeting model is to determine the minimum investment required for New York State to eliminate child care deserts. Each ZIP code must satisfy two coverage conditions: (i) a minimum number of total child care slots, and (ii) a stricter requirement for children aged 0–5. To meet these coverage thresholds, additional capacity can be created either by expanding existing facilities or by constructing new facilities.

In this Part 1 "ideal" scenario, there are no geographic siting constraints; expansions are feasible wherever capacity deficits exist. However, the **cost of expansion depends on how large the expansion is relative to the current facility size**. Specifically:

- Each facility f may add up to

$$x_f \leq \min\{1.2\, n_f,\ 500\},$$

where n_f is its current capacity.

- If the expansion remains **below the current size** (i.e. under 100%)($x_f < n_f$), the marginal cost per additional slot depends on the scale of the facility. Smaller facilities are more expensive to expand per unit (according to the given instructions), so we pick the following costs per slot:

$$c_{\text{exp}}(n_f) = \begin{cases} 200, & n_f < 100, \\ 180, & 100 \leq n_f < 300, \\ 160, & n_f \geq 300. \end{cases}$$

- If the expansion **reaches or exceeds** the original capacity ($x_f \geq n_f$), the expansion reflects a substantial restructuring effort. In that case, a fixed *baseline cost* of \$20,000 is charged, and the marginal cost becomes a uniform \$200 per slot regardless of the original size:

$$\text{Cost}_f = 200\, x_f + 20000.$$

- New facilities may be built in standardized sizes S, M, L offering $(100, 200, 400)$ total slots and $(50, 100, 200)$ age 0–5 eligible slots, at fixed construction costs (\$65,000, \$95,000, \$115,000) respectively.

- Any slot allocated to children aged 0–5 incurs an additional equipment cost of \$100 per slot (whether created via expansion or new construction).

3.1.2 Sets, Parameters, and Data

- Z: set of ZIP codes. $F_z \subseteq F$: facilities located in ZIP z.

- $S = \{S, M, L\}$: new facility sizes, each with total capacity cap_s and 0–5 capacity $\text{cap}_s^{0\text{–}5}$.

- n_f: current capacity of facility f.

- pop0_12_z, pop0_5_z: demographic data guiding capacity targets.

- Socio-economic indicators classify ZIPs into high- or normal-demand categories, affecting required coverage levels.

  - High-demand ZIPs: (either $\text{employment_rate}_z \geq 0.6$ or $\text{average_income}_z \leq 60000$):

$$\text{TotalSlots}_z \ \geq\ 0.5 \cdot \text{pop0_12}_z.$$

  - Normal-demand ZIPs:

$$\text{TotalSlots}_z \ \geq\ \tfrac{1}{3} \cdot \text{pop0_12}_z.$$

In addition to total capacity requirements, every ZIP must also satisfy an age-specific coverage requirement:

$$\text{Slots}_{0\text{–}5,z} \;\geq\; \tfrac{2}{3} \cdot \text{pop0_5}_z.$$

Thus, the socio-economic classification affects **how many total slots** must be provided, while the 0–5 requirement applies uniformly across all ZIPs.

3.1.3 Decision Variables

- $x_f \geq 0$: expansion slots added at facility f.
- $b_f \in \{0, 1\}$: indicator taking value 1 iff $x_f \geq n_f$ (major expansion regime, above 100% of initial size).
- $n_{z,s} \in \mathbb{Z}_+$: number of new facilities of size s built in ZIP z.
- $new_z^{0\text{–}5}$, $exp_z^{0\text{–}5} \geq 0$: slots allocated to ages 0–5 from construction and expansion respectively.

3.1.4 Objective Function

The expansion cost for facility f is

$$\text{Cost}_f = \begin{cases} c_{\text{exp}}(n_f)\, x_f, & x_f < n_f, \\ 200\, x_f + 20000, & x_f \geq n_f. \end{cases}$$

The model minimizes:

$$\min \quad \sum_{f \in F} \text{Cost}_f \;+\; \sum_{z,s} n_{z,s} \cdot \text{cost}_s \;+\; 100 \sum_{z \in Z} \left(new_z^{0\text{–}5} + exp_z^{0\text{–}5} \right).$$

3.1.5 Constraints

Expansion caps. Each facility may expand its capacity, but only up to the capped upper limit:

$$0 \leq x_f \leq \min\{1.2\, n_f,\ 500\}, \qquad \forall f \in F.$$

Baseline activation (indicator constraint). The binary variable b_f encodes whether the expansion reaches the scale at which the baseline cost applies:

$$b_f = 1 \;\Leftrightarrow\; x_f \geq n_f.$$

In practice, this is implemented with MILP indicator constraints to avoid nonlinearity.

ZIP-level aggregation and allocation of 0–5 slots.

$$\text{ExpTotal}_z = \sum_{f \in F_z} x_f, \qquad \forall z \in Z, \tag{1}$$

$$new_z^{0\text{–}5} \leq \sum_{s \in S} n_{z,s}\, \text{cap}_s^{0\text{–}5}, \qquad \forall z \in Z, \tag{2}$$

$$exp_z^{0\text{–}5} \leq \text{ExpTotal}_z, \qquad \forall z \in Z. \tag{3}$$

Coverage constraints (elimination of deserts). Define the minimum required capacity:

$$\text{ReqTotal}_z = \begin{cases} 0.5 \cdot \text{pop0_12}_z, & \text{if high-demand,} \\ \frac{1}{3} \cdot \text{pop0_12}_z, & \text{otherwise.} \end{cases}$$

The 0–5 coverage requirement applies uniformly:

$$\text{Req}_z^{0\text{–}5} = \tfrac{2}{3}\, \text{pop0_5}_z.$$

With CurrTotal_z and $\text{Curr}_z^{0\text{–}5}$ denoting existing capacities:

$$\text{CurrTotal}_z + \text{ExpTotal}_z + \sum_{s \in S} n_{z,s}\, \text{cap}_s \geq \text{ReqTotal}_z, \qquad \forall z \in Z, \tag{4}$$

$$\text{Curr}_z^{0\text{–}5} + exp_z^{0\text{–}5} + new_z^{0\text{–}5} \geq \text{Req}_z^{0\text{–}5}, \qquad \forall z \in Z. \tag{5}$$

Integrality and domains.

$$n_{z,s} \in \mathbb{Z}_+, \qquad x_f,\, new_z^{0\text{–}5},\, exp_z^{0\text{–}5} \geq 0, \qquad b_f \in \{0, 1\}.$$

3.1.6 Solution Approach

We implement a MILP in `gurobipy`. For each ZIP, the model computes demographic targets and enforces coverage with current capacity, expansions, and new builds. The expansion cap is linearized via the $(t1_f, t2_f, t3_f)$ segments, while the baseline fee is handled by indicator constraints on b_f. The per-slot expansion cost depends only on the initial facility size n_f, ensuring linearity in x_f; the additional \$100 applies to any 0–5 slot created through expansion or new builds.

3.2 The Problem of Realistic Capacity Expansion and Location

3.2.1 Problem Statement and Assumptions

In the realistic expansion model, we introduce two changes relative to Part 1:

- Expansion of existing facilities is limited to at most **20% of the current capacity**:

$$0 \leq x_f \leq 0.2\, n_f.$$

- The cost of expansion is **piecewise increasing** in the proportion of added capacity: small-scale renovations are relatively inexpensive, while larger redesigns become progressively more costly.

Additionally, new facilities cannot be placed too close to one another. A minimum geographic separation is required to avoid clustering of services.

3.2.2 Sets, Parameters, and Data

- $d(i, j)$: geographic distance between sites i and j.
- $D_{\min} = 0.06$ miles: minimum allowed distance between two active facilities.

All other data (populations, capacities, build options) are the same as in Part 1.

3.2.3 Decision Variables

- $t1_f, t2_f, t3_f \geq 0$: slots expanded in the intervals $[0, 10\%]$, $(10\%, 15\%]$, and $(15\%, 20\%]$ of n_f.
- $y_p \in \{1, 0\}$: a new facility is built at candidate site p if and only if $y_p = 1$.

3.2.4 Expansion Cost

$$x_f = t1_f + t2_f + t3_f,$$

with bounds:

$$0 \leq t1_f \leq 0.10\, n_f, \quad 0 \leq t2_f \leq 0.05\, n_f, \quad 0 \leq t3_f \leq 0.05\, n_f.$$

The expansion cost is:

$$\text{Cost}_f = 250\, t1_f + 400\, t2_f + 1000\, t3_f + b_f \cdot 20000,$$

where $b_f = 1$ if $x_f > 0$ (baseline fee charged when any expansion occurs).

3.2.5 Location Feasibility Constraint

For any pair of candidate sites too close to one another:

$$y_i + y_j \leq 1 \quad \text{whenever } d(i, j) < D_{\min}.$$

3.2.6 Coverage Constraints

Coverage requirements for total capacity and age 0–5 capacity remain identical to Part 1.

Handling the Spatial Feasibility and the Large Candidate Site Set

The dataset of potential construction sites contains several hundred thousand geographic points. It is not computationally feasible to include all of them directly in the optimization model. We therefore apply a two-stage spatial filtering and thinning procedure *before* formulating the MILP.

First, we restrict attention to candidate sites located in ZIP codes identified as child care deserts. Second, we eliminate any site lying within 0.06 miles of an existing facility using a `BallTree` (haversine metric) nearest-neighbor search, ensuring that new construction does not cluster around current infrastructure. Finally, within each ZIP, we apply a greedy spatial thinning algorithm: we sort candidate sites geographically and iteratively select one site while discarding all neighboring sites within 0.06 miles. This guarantees that all remaining candidate sites are mutually spaced by at least the required distance.

As a result, the optimization model receives only a small, spatially feasible subset of candidate locations, avoiding the need to impose $O(N^2)$ pairwise distance constraints. This reduces the location problem from hundreds of thousands of potential sites to a few feasible options per ZIP, making the MILP formulation tractable while strictly enforcing the minimum-site-spacing requirement.

> ***BallTree nearest-neighbor filtering*** *The BallTree is a data structure used to store points in space so that nearest-neighbor searches can be done very quickly. Instead of comparing a new point to every existing facility (which would be too slow with thousands of sites), the BallTree groups nearby points together into "balls" (spheres) in a hierarchical structure. To find the closest facility to a candidate location, the search algorithm looks only at the balls that could possibly contain a close neighbor and ignores the others. This reduces the search time from checking all facilities to checking only a small subset. We use this to test each candidate site: if its nearest existing facility is within* 0.06 *miles, the candidate is removed. This approach makes the distance filtering step fast and scalable, even when starting from very large spatial datasets.*

3.2.7 Objective

$$\min \quad \sum_{f \in F} \text{Cost}_f + \sum_{z,s} n_{z,s} \cdot \text{cost}_s + 100 \sum_{z \in Z} (new_z^{0\text{–}5} + exp_z^{0\text{–}5}).$$

4 Results and Conclusions

We evaluate three scenarios that differ in how flexible expansions are and whether spatial feasibility is enforced. The total statewide cost required to eliminate child care deserts changes significantly

depending on these assumptions.

Table 1: Optimal statewide funding required under each modeling scenario.

Scenario	Optimal Cost ($)
Part 1 — Ideal	348,568,508.81
Part 1 — Realistic (20% expansion cap)	519,250,157.09
Part 2 — Realistic (+ spatial feasibility)	391,626,999.00

Interpretation

The **Part 1 Ideal** scenario is the least expensive because it allows facilities to expand by up to 120% of their current size. This lets the model add many new slots at locations where infrastructure and staffing already exist, which is cheaper than building new facilities.

In the **Part 1 Realistic** scenario, expansions are limited to only 20% of current capacity and become more expensive as they get closer to this limit. As a result, the model must rely much more on constructing new facilities, which drives the cost up by nearly **50%**. This shows that **expansion flexibility is one of the biggest levers in reducing total cost**.

Interestingly, introducing the **spatial distance constraint** in **Part 2 Realistic** reduces the optimal cost from $519M to about $392M. This may seem counter-intuitive at first, because adding a constraint often increases cost. However, the spatial filtering step removes candidates that are too close to existing facilities or to each other. This has the effect of **nudging the model toward fewer, larger, and better-placed facilities** instead of many small, scattered ones. Fewer construction decisions = lower total cost.
In the Part 1 Realistic model, facility placement decisions are made independently at the ZIP level, without considering geographic proximity, so the model may construct many small facilities clustered very close to one another, which satisfies coverage constraints locally but leads to a higher overall cost because capacity is not spatially consolidated.

In other words, the spacing rule implicitly encourages more **coordinated, consolidated capacity planning**, which turns out to be more cost-efficient.

Key Takeaways

- Allowing large expansions reduces cost substantially by leveraging existing infrastructure.
- Limiting expansions forces the model to build more facilities, strongly increasing cost.
- Introducing reasonable geographic spacing *can* reduce cost, because it encourages fewer but more impactful construction decisions.
- The most important cost driver is not distance, but **how much expansion is allowed per facility**.

5 Fairness Considerations (Optional Extension)

Beyond eliminating child care deserts, the state may also wish to ensure that access to child care is *fairly distributed* across regions. Eliminating deserts ensures that every ZIP receives a *minimum* level of access, but it does not prevent some areas from having substantially more access than others. To capture this equity dimension, we evaluate the **weighted coverage ratio** for each ZIP z:

$$\text{cov}_z = \tfrac{2}{3} \cdot \frac{\text{Slots}_{0-5,z}}{\text{Population}_{0-5,z}} + \tfrac{1}{3} \cdot \frac{\text{Slots}_{0-12,z}}{\text{Population}_{0-12,z}}.$$

This weighting reflects the higher social importance of care access in early childhood. The statewide **fairness gap** is then defined as:

$$\text{gap} = \max_{z \in Z} \text{cov}_z - \min_{z \in Z} \text{cov}_z.$$

A smaller gap corresponds to a more equitable distribution of care. Policy discussions often target gap ≤ 0.1, meaning access levels should not differ by more than ten percentage points across ZIPs.

Optimization Goal

We analyze fairness under two different policy settings:

- **Phase A (Budget-Constrained):** Improve fairness as much as possible under a hard budget limit of \$100 million.
- **Phase B (Fairness-Constrained):** Achieve the fairness target gap ≤ 0.1 at the minimum possible cost, without a budget cap.

Phase A — Minimizing Fairness Gap With a \$100M Budget

The optimization in this phase uses a **lexicographic objective**:

1. Minimize the fairness gap.
2. Subject to that minimized gap, maximize total cost used (up to \$100M).

As the results show in the Notebook, it is not possible to reach the state objective with such a budget.

Interpretation. Even when the entire budget is spent optimally, the fairness gap remains above 0.1

Phase B — Achieving the Fairness Target (No Budget Cap)

In this phase, we remove the budget constraint and try to assess the minimum required to budget to reach our following objective:

$$\text{gap} \leq 0.10,$$

This tells us how much the state would need to invest to ensure that all ZIPs enjoy nearly equal child care access.

Metric	**Phase B Result**
Achieved fairness gap	0.1
Minimum cost required	$\approx$ \$243.5 millions

Interpretation. Achieving equitable access statewide would require approximately \$ **243.5 millions**. This figure is $\approx$ 2.4 times larger than the funds allocated in this program, illustrating that:

- Closing long-standing geographic access inequalities is vastly more expensive.
- Fairness requires not only incremental expansion but large-scale structural capacity redistribution.

Conclusion

This fairness extension highlights an important policy tradeoff:

$$\text{Eliminating deserts} \neq \text{Achieving equal access.}$$

The state can ensure basic child care availability with moderate investment, but **true equity in access requires a transformative level of funding**. Policymakers must therefore decide how to balance cost-efficiency and fairness when planning long-term support for child care infrastructure.

Note. The total cost observed in this fairness-focused model is lower than in Parts 1 and 2 because we no longer enforce the requirement that all child care deserts must be fully eliminated. The model is therefore allowed to leave some high-cost areas partially underserved, which reduces the overall expenditure while improving fairness according to the weighted coverage objective.

6 Future Work: Toward a Combined Objective

While the two-phase approach separately optimizes for cost efficiency and fairness, real-world policy decisions rarely prioritize one objective exclusively. A promising direction for future research would be to formulate a **convex combination of both objectives**—minimizing a weighted sum of total

cost and fairness gap:

$$\boxed{\min \quad \alpha \cdot \text{Cost} + (1-\alpha) \cdot \text{Gap} \quad \text{where } \alpha \in [0,1]}$$

This formulation explicitly encodes the tradeoff between equity and efficiency. By adjusting the weight α, policymakers can explore an entire Pareto frontier of solutions—ranging from low-cost but potentially unequal allocations ($\alpha \to 1$) to highly equitable but expensive plans ($\alpha \to 0$). Such a unified model would make it possible to quantify, in monetary terms, **how much additional funding is required to achieve a given level of fairness**. It also provides a natural framework for sensitivity analysis, helping decision-makers balance short-term budget constraints with long-term social equity goals.

7 Appendix (Code)

Part1_Ideal

October 28, 2025

[1]:
```
from gurobipy import Model, GRB, quicksum
import pandas as pd
import numpy as np
import os
```

[2]:
```
income_df = pd.read_csv("./ChildCareDeserts_Data/avg_individual_income.csv")
employment_df = pd.read_csv("./ChildCareDeserts_Data/employment_rate.csv")
population_df = pd.read_csv("./ChildCareDeserts_Data/population.csv")
childcare_df = pd.read_csv("./ChildCareDeserts_Data/child_care_regulated.csv")
location_df = pd.read_csv("./ChildCareDeserts_Data/potential_locations.csv")
```

[3]:
```
# ---------- Helpers ----------
def normalize_zip(df):
    for col in df.columns:
        if "zip" in col.lower():
            df = df.rename(columns={col: "ZIP"})
            break
    df["ZIP"] = df["ZIP"].astype(str).str.zfill(5)
    return df


def first_col(df, keys):
    for c in df.columns:
        lc = c.lower()
        if any(k in lc for k in keys):
            return c
    return None
```

[4]:
```
# ---------- normalize zips ----------
income_df = normalize_zip(income_df)
employment_df = normalize_zip(employment_df)
population_df = normalize_zip(population_df)
childcare_df = normalize_zip(childcare_df)
location_df = normalize_zip(location_df)
```

[5]:
```
# ---------- current capacity per facility & per ZIP ----------
if "total_capacity" in childcare_df.columns:
    childcare_df["fac_capacity"] = childcare_df["total_capacity"]
```

```
else:
    cap_cols = [c for c in childcare_df.columns if "capacity" in c.lower()]
    if len(cap_cols) == 0:
        raise ValueError("No capacity columns found in child_care_regulated.
 ↪csv")
    childcare_df["fac_capacity"] = childcare_df[cap_cols].sum(axis=1,␣
 ↪numeric_only=True)
```

[6]:
```
# Best-effort existing 0-5 capacity (if present)
u5_cols = [
    c
    for c in childcare_df.columns
    if any(k in c.lower() for k in ["0-5", "0-5", "infant", "toddler",␣
 ↪"preschool"])
]
if len(u5_cols) > 0:
    childcare_df["fac_u5_capacity"] = childcare_df[u5_cols].sum(
        axis=1, numeric_only=True
    )
else:
    childcare_df["fac_u5_capacity"] = 0.0
```

[7]:
```
# Per-ZIP existing totals
zip_capacity = (
    childcare_df.groupby("ZIP", as_index=False)["fac_capacity"]
    .sum()
    .rename(columns={"fac_capacity": "current_capacity"})
)
zip_u5_capacity = (
    childcare_df.groupby("ZIP", as_index=False)["fac_u5_capacity"]
    .sum()
    .rename(columns={"fac_u5_capacity": "current_u5_capacity"})
)
```

[8]:
```
# ---------- population (0-5 and 0-12) ----------
# Try to find typical buckets: '0-5','5-9','10-14'
c_0_5 = [
    c
    for c in population_df.columns
    if "0-5" in c.lower() or "0-5" in c.lower() or "under 5" in c.lower()
]
c_5_9 = [c for c in population_df.columns if "5-9" in c.lower() or "5-9" in c.
 ↪lower()]
c_10_14 = [
    c for c in population_df.columns if "10-14" in c.lower() or "10-14" in c.
 ↪lower()
]
```

```python
def safe_sum_cols(df, cols):
    return df[cols].sum(axis=1, numeric_only=True) if len(cols) > 0 else 0.0


population_df["pop_0_5"] = safe_sum_cols(population_df, c_0_5)
population_df["pop_5_9"] = safe_sum_cols(population_df, c_5_9)
population_df["pop_10_14"] = safe_sum_cols(population_df, c_10_14)

# 0-12 = (0-5) + (5-9) + (2/5)*(10-14)
population_df["pop_0_12"] = (
    population_df["pop_0_5"]
    + population_df["pop_5_9"]
    + 0.4 * population_df["pop_10_14"]
)

zip_population = population_df[["ZIP", "pop_0_5", "pop_0_12"]].copy()
```

[9]:
```python
# ---------- employment / income ----------
employment_df = employment_df.rename(columns={"employment rate":␣
 ↪"employment_rate"})
income_df = income_df.rename(columns={"average income": "average_income"})

# ---------- per-ZIP base table ----------
zip_df = location_df[["ZIP"]].drop_duplicates()
zip_df = zip_df.merge(zip_capacity, on="ZIP", how="left")
zip_df = zip_df.merge(zip_u5_capacity, on="ZIP", how="left")
zip_df = zip_df.merge(zip_population, on="ZIP", how="left")
zip_df = zip_df.merge(employment_df[["ZIP", "employment_rate"]], on="ZIP",␣
 ↪how="left")
zip_df = zip_df.merge(income_df[["ZIP", "average_income"]], on="ZIP",␣
 ↪how="left")
zip_df[
    [
        "current_capacity",
        "current_u5_capacity",
        "pop_0_5",
        "pop_0_12",
        "employment_rate",
        "average_income",
    ]
] = zip_df[
    [
        "current_capacity",
        "current_u5_capacity",
        "pop_0_5",
```

```
        "pop_0_12",
        "employment_rate",
        "average_income",
    ]
].fillna(0)

ZIPs = zip_df["ZIP"].tolist()
```

[10]:
```
# ---------- facility options (Problem 1) ----------
facility_options = [
    {"name": "S", "cap_total": 100, "cap_05_max": 50, "cost": 65000},
    {"name": "M", "cap_total": 200, "cap_05_max": 100, "cost": 95000},
    {"name": "L", "cap_total": 400, "cap_05_max": 200, "cost": 115000},
]
```

[11]:
```
# ---------- MODEL ----------
m = Model("MinCostChildcare_P1")

# --- NEW FACILITIES: integer count per ZIP and size (Problem 1 allows any␣
↪number anywhere) ---
n_build = {
    (z, opt["name"]): m.addVar(vtype=GRB.INTEGER, lb=0,␣
↪name=f"build_{z}_{opt['name']}")
    for z in ZIPs
    for opt in facility_options
}

# Total new capacity and new under-5 capacity per ZIP
new_total = {
    z: quicksum(
        n_build[(z, opt["name"])] * opt["cap_total"] for opt in facility_options
    )
    for z in ZIPs
}
# decision variable for how many of the new slots are assigned to 0-5, capped␣
↪by per-size max
new_u5 = {z: m.addVar(lb=0, vtype=GRB.CONTINUOUS, name=f"new_u5_{z}") for z in␣
↪ZIPs}
for z in ZIPs:
    cap_u5_max = quicksum(
        n_build[(z, opt["name"])] * opt["cap_05_max"] for opt in␣
↪facility_options
    )
    m.addConstr(new_u5[z] <= cap_u5_max, name=f"u5_new_cap_{z}")
```

```
Set parameter Username
Set parameter LicenseID to value 2722177
```

```
Academic license - for non-commercial use only - expires 2026-10-14
```

[ ]:
```
# --- EXPANSION: per existing facility f (cap min(20%·n_f, 500)) ---
childcare_df = (
    childcare_df.reset_index(drop=True).reset_index().rename(columns={"index":␣
 ↪"f_id"})
)
F = childcare_df["f_id"].tolist()
fac_zip = dict(zip(childcare_df["f_id"], childcare_df["ZIP"]))
n_f = dict(zip(childcare_df["f_id"], childcare_df["fac_capacity"]))

# ---------- EXPANSION  ----------
t1, t2, t3, x_f = {}, {}, {}, {}
b_base = {}

for f in F:
    cap120 = 1.2 * n_f[f]
    caplim = min(cap120, 500.0)  # max 120% ou +500 slots

    # segment limits: 0-50%, 50-100%, 100-120% (approximation linéaire)
    t1_ub = min(0.50 * n_f[f], caplim)
    t2_ub = min(max(0.0, 1.00 * n_f[f] - 0.50 * n_f[f]), max(0.0, caplim -␣
 ↪t1_ub))
    t3_ub = max(0.0, caplim - t1_ub - t2_ub)

    t1[f] = m.addVar(lb=0, ub=t1_ub, vtype=GRB.CONTINUOUS, name=f"t1_{f}")
    t2[f] = m.addVar(lb=0, ub=t2_ub, vtype=GRB.CONTINUOUS, name=f"t2_{f}")
    t3[f] = m.addVar(lb=0, ub=t3_ub, vtype=GRB.CONTINUOUS, name=f"t3_{f}")
    x_f[f] = m.addVar(lb=0, ub=caplim, vtype=GRB.CONTINUOUS, name=f"x_{f}")
    b_base[f] = m.addVar(vtype=GRB.BINARY, name=f"base_{f}")

    m.addConstr(x_f[f] == t1[f] + t2[f] + t3[f], name=f"x_sum_{f}")

    m.addGenConstrIndicator(b_base[f], 1, x_f[f] >= n_f[f], name=f"ind_ge_{f}")
    m.addGenConstrIndicator(b_base[f], 0, x_f[f] <= n_f[f] - 1e-6,␣
 ↪name=f"ind_lt_{f}")

exp_total = {z: quicksum(x_f[f] for f in F if fac_zip[f] == z) for z in ZIPs}

exp_u5 = {z: m.addVar(lb=0, vtype=GRB.CONTINUOUS, name=f"exp_u5_{z}") for z in␣
 ↪ZIPs}
for z in ZIPs:
    m.addConstr(exp_u5[z] <= exp_total[z], name=f"exp_u5_le_exp_{z}")
```

[13]:
```
# ---------- COVERAGE CONSTRAINTS ----------
EMPLOYMENT_T = 60
INCOME_T = 60000
```

```
zip_idx = {z: i for i, z in enumerate(ZIPs)}
zip_series = zip_df.set_index("ZIP")

for z in ZIPs:
    employ_rate = float(zip_series.loc[z, "employment_rate"])
    avg_income = float(zip_series.loc[z, "average_income"])
    pop_012 = float(zip_series.loc[z, "pop_0_12"])
    pop_05 = float(zip_series.loc[z, "pop_0_5"])
    curr_total = float(zip_series.loc[z, "current_capacity"])
    curr_u5 = float(zip_series.loc[z, "current_u5_capacity"])

    is_high = (employ_rate >= EMPLOYMENT_T) or (avg_income <= INCOME_T)
    required_total = 0.5 * pop_012 if is_high else (1.0 / 3.0) * pop_012

    # Total coverage (0-12)
    m.addConstr(
        curr_total + exp_total[z] + new_total[z] >= required_total,
        name=f"cov_total_{z}",
    )

    # 0-5 coverage ( 2/3 of 0-5 population)
    m.addConstr(
        curr_u5 + exp_u5[z] + new_u5[z] >= (2.0 / 3.0) * pop_05,␣
 ↪name=f"cov_u5_{z}"
    )
```

[14]:
```
# ---------- COSTS  ----------
# Ensure indicator meaning:
# b_base[f] = 1     x_f >= n_f
for f in F:
    m.addGenConstrIndicator(b_base[f], True, x_f[f] >= n_f[f])
    m.addGenConstrIndicator(b_base[f], False, x_f[f] <= n_f[f] - 1e-6)


# Cost per slot BELOW 100% (depends on initial facility size)
def cost_per_slot_low(n):
    if n < 100:
        return 200
    elif n < 300:
        return 180
    else:
        return 160


expansion_cost_low = quicksum(
```

```
    cost_per_slot_low(n_f[f]) * x_f[f] * (1 - b_base[f])  # only applies when␣
 ↪x_f < n_f
    for f in F
)

# Cost per slot ABOVE 100% (always 200 per slot)
expansion_cost_high = quicksum(
    200.0 * x_f[f] * b_base[f]  # only applies when x_f >= n_f
    for f in F
)

# Baseline fee triggered only if x_f >= n_f (fixed 20,000)
baseline_cost = quicksum(b_base[f] * 20000.0 for f in F)

# New builds (unchanged)
build_cost = quicksum(
    n_build[(z, opt["name"])] * opt["cost"] for z in ZIPs for opt in␣
 ↪facility_options
)

# Cost for new 0-5 slots (unchanged)
equip_cost = quicksum(100.0 * (new_u5[z] + exp_u5[z]) for z in ZIPs)

# Final objective
m.setObjective(
    expansion_cost_low + expansion_cost_high + baseline_cost + build_cost +␣
 ↪equip_cost,
    GRB.MINIMIZE,
)
```

[15]:
```
# ---------- SOLVE ----------
m.optimize()

if m.status == GRB.OPTIMAL:
    print(f"\nOptimal total cost: ${m.objVal:,.2f}\n")
```

```
Gurobi Optimizer version 12.0.3 build v12.0.3rc0 (mac64[arm] - Darwin 24.6.0
24G231)

CPU model: Apple M3
Thread count: 8 physical cores, 8 logical processors, using up to 8 threads

Optimize a model with 24220 rows, 88790 columns and 114278 nonzeros
Model fingerprint: 0xfd72345a
Model has 2051 quadratic objective terms
Model has 62416 simple general constraints
  62416 INDICATOR
```

```
Variable types: 66724 continuous, 22066 integer (15604 binary)
Coefficient statistics:
  Matrix range     [1e+00, 4e+02]
  Objective range  [1e+02, 1e+05]
  QObjective range [4e+01, 8e+01]
  Bounds range     [6e-01, 5e+02]
  RHS range        [3e-01, 6e+03]
  GenCon rhs range [1e-06, 9e+02]
  GenCon coe range [1e+00, 1e+00]
Presolve added 61930 rows and 0 columns
Presolve removed 0 rows and 4917 columns
Presolve time: 0.30s
Presolved: 88186 rows, 85909 columns, 202300 nonzeros
Presolved model has 824 SOS constraint(s)
Variable types: 17136 continuous, 68773 integer (66077 binary)
Found heuristic solution: objective 8.452347e+08
Found heuristic solution: objective 8.451728e+08
Found heuristic solution: objective 5.929729e+08
Performing another presolve…
Presolve removed 88172 rows and 85883 columns
Presolve time: 0.72s
Found heuristic solution: objective 3.486863e+08

Root relaxation: objective 3.485123e+08, 6 iterations, 0.00 seconds (0.00 work
units)

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0 3.4851e+08    0    1 3.4869e+08 3.4851e+08  0.05%     -    1s
H    0     0                     3.485985e+08 3.4851e+08  0.02%     -    1s
H    0     0                     3.485785e+08 3.4851e+08  0.02%     -    1s
H    0     0                     3.485685e+08 3.4851e+08  0.02%     -    1s

Cutting planes:
  MIR: 1

Explored 1 nodes (6 simplex iterations) in 1.28 seconds (1.25 work units)
Thread count was 8 (of 8 available processors)

Solution count 7: 3.48569e+08 3.48579e+08 3.48686e+08 … 8.45235e+08

Optimal solution found (tolerance 1.00e-04)
Warning: max constraint violation (1.2600e-04) exceeds tolerance
Warning: max general constraint violation (1.2600e-04) exceeds tolerance
Best objective 3.485685088079e+08, best bound 3.485485088305e+08, gap 0.0057%

Optimal total cost: $348,568,508.81
```

Part1_Realistic

October 28, 2025

1 Part 1 - Reaslistic

In the realistic scenario, additional constraints make the problem more complex and closer to real-life conditions. The key differences are:

Limited expansion: Due to space constraints, no facility can expand beyond 20% of its current capacity. This replaces the 120%/+500 slot allowance from Part 1.

Tiered expansion costs: The cost to expand an existing facility increases with the scale of expansion:

0–10% increase: $20,000 + $200 per existing slot (baseline low cost)

10–15% increase: $20,000 + $400 per existing slot (higher cost)

15–20% increase: $20,000 + $1000 per existing slot (highest cost)

[4]:
```
from gurobipy import Model, GRB, quicksum
import pandas as pd
import numpy as np
import os

income_df = pd.read_csv("./ChildCareDeserts_Data/avg_individual_income.csv")
employment_df = pd.read_csv("./ChildCareDeserts_Data/employment_rate.csv")
population_df = pd.read_csv("./ChildCareDeserts_Data/population.csv")
childcare_df = pd.read_csv("./ChildCareDeserts_Data/child_care_regulated.csv")
childcare_df = pd.read_csv("./ChildCareDeserts_Data/potential_locations.csv")


# ---------- Helpers ----------
def normalize_zip(df):
    for col in df.columns:
        if "zip" in col.lower():
            df = df.rename(columns={col: "ZIP"})
            break
    df["ZIP"] = df["ZIP"].astype(str).str.zfill(5)
    return df


def first_col(df, keys):
    for c in df.columns:
```

```
        lc = c.lower()
        if any(k in lc for k in keys):
            return c
    return None


# ---------- normalize zips ----------
income_df = normalize_zip(income_df)
employment_df = normalize_zip(employment_df)
population_df = normalize_zip(population_df)
childcare_df = normalize_zip(childcare_df)
childcare_df = normalize_zip(childcare_df)

# ---------- current capacity per facility & per ZIP ----------
if "total_capacity" in childcare_df.columns:
    childcare_df["fac_capacity"] = childcare_df["total_capacity"]
else:
    cap_cols = [c for c in childcare_df.columns if "capacity" in c.lower()]
    if len(cap_cols) == 0:
        raise ValueError("No capacity columns found in child_care_regulated.
 ↪csv")
    childcare_df["fac_capacity"] = childcare_df[cap_cols].sum(axis=1,␣
 ↪numeric_only=True)
u5_cols = [
    c
    for c in childcare_df.columns
    if any(k in c.lower() for k in ["0-5", "0-5", "infant", "toddler",␣
 ↪"preschool"])
]
if len(u5_cols) > 0:
    childcare_df["fac_u5_capacity"] = childcare_df[u5_cols].sum(
        axis=1, numeric_only=True
    )
else:
    childcare_df["fac_u5_capacity"] = 0.0
zip_capacity = (
    childcare_df.groupby("ZIP", as_index=False)["fac_capacity"]
    .sum()
    .rename(columns={"fac_capacity": "current_capacity"})
)
zip_u5_capacity = (
    childcare_df.groupby("ZIP", as_index=False)["fac_u5_capacity"]
    .sum()
    .rename(columns={"fac_u5_capacity": "current_u5_capacity"})
)
# ---------- population (0-5 and 0-12) ----------
c_0_5 = [
```

```
    c
    for c in population_df.columns
    if "0-5" in c.lower() or "0-5" in c.lower() or "under 5" in c.lower()
]
c_5_9 = [c for c in population_df.columns if "5-9" in c.lower() or "5-9" in c.
 ↪lower()]
c_10_14 = [
    c for c in population_df.columns if "10-14" in c.lower() or "10-14" in c.
 ↪lower()
]


def safe_sum_cols(df, cols):
    return df[cols].sum(axis=1, numeric_only=True) if len(cols) > 0 else 0.0


population_df["pop_0_5"] = safe_sum_cols(population_df, c_0_5)
population_df["pop_5_9"] = safe_sum_cols(population_df, c_5_9)
population_df["pop_10_14"] = safe_sum_cols(population_df, c_10_14)

# 0-12 = (0-5) + (5-9) + (2/5)*(10-14)
population_df["pop_0_12"] = (
    population_df["pop_0_5"]
    + population_df["pop_5_9"]
    + 0.4 * population_df["pop_10_14"]
)

zip_population = population_df[["ZIP", "pop_0_5", "pop_0_12"]].copy()
# ---------- employment / income ----------
employment_df = employment_df.rename(columns={"employment rate":␣
 ↪"employment_rate"})
income_df = income_df.rename(columns={"average income": "average_income"})

# ---------- per-ZIP base table ----------
zip_df = childcare_df[["ZIP"]].drop_duplicates()
zip_df = zip_df.merge(zip_capacity, on="ZIP", how="left")
zip_df = zip_df.merge(zip_u5_capacity, on="ZIP", how="left")
zip_df = zip_df.merge(zip_population, on="ZIP", how="left")
zip_df = zip_df.merge(employment_df[["ZIP", "employment_rate"]], on="ZIP",␣
 ↪how="left")
zip_df = zip_df.merge(income_df[["ZIP", "average_income"]], on="ZIP",␣
 ↪how="left")
zip_df[
    [
        "current_capacity",
        "current_u5_capacity",
        "pop_0_5",
```

```
        "pop_0_12",
        "employment_rate",
        "average_income",
    ]
] = zip_df[
    [
        "current_capacity",
        "current_u5_capacity",
        "pop_0_5",
        "pop_0_12",
        "employment_rate",
        "average_income",
    ]
].fillna(0)

ZIPs = zip_df["ZIP"].tolist()
# ---------- facility options (Problem 1) ----------
facility_options = [
    {"name": "S", "cap_total": 100, "cap_05_max": 50, "cost": 65000},
    {"name": "M", "cap_total": 200, "cap_05_max": 100, "cost": 95000},
    {"name": "L", "cap_total": 400, "cap_05_max": 200, "cost": 115000},
]
# ---------- MODEL ----------
m = Model("MinCostChildcare_P1")

# --- NEW FACILITIES: integer count per ZIP and size ---
n_build = {
    (z, opt["name"]): m.addVar(vtype=GRB.INTEGER, lb=0,␣
 ↪name=f"build_{z}_{opt['name']}")
    for z in ZIPs
    for opt in facility_options
}

# Total new capacity and new under-5 capacity per ZIP
new_total = {
    z: quicksum(
        n_build[(z, opt["name"])] * opt["cap_total"] for opt in facility_options
    )
    for z in ZIPs
}
new_u5 = {z: m.addVar(lb=0, vtype=GRB.CONTINUOUS, name=f"new_u5_{z}") for z in␣
 ↪ZIPs}
for z in ZIPs:
    cap_u5_max = quicksum(
        n_build[(z, opt["name"])] * opt["cap_05_max"] for opt in␣
 ↪facility_options
    )
```

```
    m.addConstr(new_u5[z] <= cap_u5_max, name=f"u5_new_cap_{z}")

# --- EXPANSION: per existing facility f (cap min(20%·n_f, 500)) ---
childcare_df = (
    childcare_df.reset_index(drop=True).reset_index().rename(columns={"index":␣
 ↪"f_id"})
)
F = childcare_df["f_id"].tolist()
fac_zip = dict(zip(childcare_df["f_id"], childcare_df["ZIP"]))
n_f = dict(zip(childcare_df["f_id"], childcare_df["fac_capacity"]))

# ---------- EXPANSION ----------
t1, t2, t3, x_f = {}, {}, {}, {}

for f in F:
    caplim = 0.20 * n_f[f]

    t1_ub = 0.10 * n_f[f]
    t2_ub = 0.05 * n_f[f]
    t3_ub = 0.05 * n_f[f]

    t1[f] = m.addVar(lb=0, ub=t1_ub, vtype=GRB.CONTINUOUS, name=f"t1_{f}")
    t2[f] = m.addVar(lb=0, ub=t2_ub, vtype=GRB.CONTINUOUS, name=f"t2_{f}")
    t3[f] = m.addVar(lb=0, ub=t3_ub, vtype=GRB.CONTINUOUS, name=f"t3_{f}")
    x_f[f] = m.addVar(lb=0, ub=caplim, vtype=GRB.CONTINUOUS, name=f"x_{f}")

    m.addConstr(x_f[f] == t1[f] + t2[f] + t3[f], name=f"x_sum_{f}")

# aggregate expansion by ZIP
exp_total = {z: quicksum(x_f[f] for f in F if fac_zip[f] == z) for z in ZIPs}

# allow some expansion to count toward 0-5 (if you prefer: set exp_u5 = 0 to be␣
 ↪conservative)
exp_u5 = {z: m.addVar(lb=0, vtype=GRB.CONTINUOUS, name=f"exp_u5_{z}") for z in␣
 ↪ZIPs}
for z in ZIPs:
    m.addConstr(exp_u5[z] <= exp_total[z], name=f"exp_u5_le_exp_{z}")

# ---------- COVERAGE CONSTRAINTS ----------
EMPLOYMENT_T = 60
INCOME_T = 60000

zip_idx = {z: i for i, z in enumerate(ZIPs)}
zip_series = zip_df.set_index("ZIP")

for z in ZIPs:
    employ_rate = float(zip_series.loc[z, "employment_rate"])
```

```
    avg_income = float(zip_series.loc[z, "average_income"])
    pop_012 = float(zip_series.loc[z, "pop_0_12"])
    pop_05 = float(zip_series.loc[z, "pop_0_5"])
    curr_total = float(zip_series.loc[z, "current_capacity"])
    curr_u5 = float(zip_series.loc[z, "current_u5_capacity"])

    is_high = (employ_rate >= EMPLOYMENT_T) or (avg_income <= INCOME_T)
    required_total = 0.5 * pop_012 if is_high else (1.0 / 3.0) * pop_012

    # Total coverage (0-12)
    m.addConstr(
        curr_total + exp_total[z] + new_total[z] >= required_total,
        name=f"cov_total_{z}",
    )

    # 0-5 coverage ( 2/3 of 0-5 population)
    m.addConstr(
        curr_u5 + exp_u5[z] + new_u5[z] >= (2.0 / 3.0) * pop_05,␣
 ↪name=f"cov_u5_{z}"
    )
# ---------- COSTS ----------

expansion_cost_terms = []

for f in F:
    cap = n_f[f]
    if cap <= 0:
        continue
    expansion_cost_terms.append(
        (20000.0 + 200.0 * cap) * (t1[f] / cap)
        + (20000.0 + 400.0 * cap) * (t2[f] / cap)
        + (20000.0 + 1000.0 * cap) * (t3[f] / cap)
    )
expansion_cost = quicksum(expansion_cost_terms)


build_cost = quicksum(
    n_build[(z, opt["name"])] * opt["cost"] for z in ZIPs for opt in␣
 ↪facility_options
)

equip_cost = quicksum(100.0 * (new_u5[z] + exp_u5[z]) for z in ZIPs)

m.setObjective(expansion_cost + build_cost + equip_cost, GRB.MINIMIZE)
# ---------- SOLVE ----------
m.optimize()
```

```python
if m.status == GRB.OPTIMAL:
    print(f"\nOptimal total cost: ${m.objVal:,.2f}\n")
```

```
Gurobi Optimizer version 12.0.3 build v12.0.3rc0 (mac64[arm] - Darwin 24.6.0
24G231)

CPU model: Apple M3
Thread count: 8 physical cores, 8 logical processors, using up to 8 threads

Optimize a model with 24220 rows, 73186 columns and 114278 nonzeros
Model fingerprint: 0x000b4c9f
Variable types: 66724 continuous, 6462 integer (0 binary)
Coefficient statistics:
  Matrix range     [1e+00, 4e+02]
  Objective range  [1e+02, 1e+05]
  Bounds range     [2e-01, 2e+02]
  RHS range        [3e-01, 6e+03]
Found heuristic solution: objective 5.818821e+08
Presolve removed 24219 rows and 73072 columns
Presolve time: 0.22s
Presolved: 1 rows, 114 columns, 114 nonzeros
Found heuristic solution: objective 5.194033e+08
Variable types: 111 continuous, 3 integer (0 binary)

Root relaxation: objective 5.192024e+08, 1 iterations, 0.00 seconds (0.00 work
units)

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0 5.1920e+08    0    1 5.1940e+08 5.1920e+08  0.04%     -    0s
H    0     0                     5.192833e+08 5.1920e+08  0.02%     -    0s
H    0     0                     5.192633e+08 5.1920e+08  0.01%     -    0s
H    0     0                     5.192502e+08 5.1920e+08  0.01%     -    0s

Explored 1 nodes (1 simplex iterations) in 0.24 seconds (0.17 work units)
Thread count was 8 (of 8 available processors)

Solution count 5: 5.1925e+08 5.19263e+08 5.19283e+08 … 5.81882e+08

Optimal solution found (tolerance 1.00e-04)
Best objective 5.192501570902e+08, best bound 5.192024177627e+08, gap 0.0092%

Optimal total cost: $519,250,157.09
```

Part2_Realistic

October 28, 2025

1 Part 2 - Realistic

Major difference with part 1 concern the distance constraint :

Distance constraints: To avoid over-concentration, any two facilities in the same area must be at least 0.06 miles apart. This means we cannot build new centers too close to existing ones or to each other within a zip code. We will use the provided potential_locations.csv to ensure new facilities are placed at valid locations respecting this distance.

Below is the Python script for Part 2. It reuses much of Part 1's setup but adjusts expansion limits and costs, and incorporates the distance constraints. We also enforce that the number of new facilities built in each zip does not exceed the number of available potential locations that satisfy the 0.06 mile spacing (after filtering out those too close to existing centers). The model then finds the minimum funding required under these realistic conditions.

[ ]:
```
import pandas as pd
import numpy as np
import math
import gurobipy as gp
from gurobipy import GRB
from sklearn.neighbors import BallTree

# 1. Load and preprocess
# --------------------------
population_df = pd.read_csv("./ChildCareDeserts_Data/population.csv")
income_df = pd.read_csv("./ChildCareDeserts_Data/avg_individual_income.csv").
 ↪set_index(
    "ZIP code"
)
employment_df = pd.read_csv("./ChildCareDeserts_Data/employment_rate.csv").
 ↪set_index(
    "zipcode"
)
childcare_df = pd.read_csv("./ChildCareDeserts_Data/child_care_regulated.csv")

location_df = pd.read_csv("./ChildCareDeserts_Data/potential_locations.csv")


population_df.set_index("zipcode", inplace=True)
```

```
# Compute number of children under 5 and number of children 0-12 in each ZIP.
population_df["children_under5"] = population_df["-5"]
population_df["children_0_12"] = (
    population_df["-5"] + population_df["5-9"] + 0.6 * population_df["10-14"]
)
population_df["children_0_12"] = population_df["children_0_12"].round()

population_df = population_df.join(income_df["average income"], how="left").
 ↪join(
    employment_df["employment rate"], how="left"
)
population_df["average income"].fillna(
    population_df["average income"].median(), inplace=True
)
population_df["employment rate"].fillna(
    population_df["employment rate"].median(), inplace=True
)


population_df["high_demand"] = (population_df["employment rate"] >= 0.60) | (
    population_df["average income"] <= 60000
)

# 2. Determine current child care capacity per ZIP
# ------------------------------------------------
under5_caps = []
total_caps = []
for idx, row in childcare_df.iterrows():
    if row["children_capacity"] and row["children_capacity"] > 0:
        under5 = row["children_capacity"]
    else:
        under5 = (
            (0 if np.isnan(row["infant_capacity"]) else row["infant_capacity"])
            + (0 if np.isnan(row["toddler_capacity"]) else␣
 ↪row["toddler_capacity"])
            + (0 if np.isnan(row["preschool_capacity"]) else␣
 ↪row["preschool_capacity"])
        )
    if row["total_capacity"] and row["total_capacity"] > 0:
        total = row["total_capacity"]
    else:
        school = (
            0 if np.isnan(row["school_age_capacity"]) else␣
 ↪row["school_age_capacity"]
        )
        total = under5 + school
```

```
    under5_caps.append(under5)
    total_caps.append(total)
childcare_df["under5_capacity"] = under5_caps
childcare_df["total_capacity_computed"] = total_caps

current_supply = childcare_df.groupby("zip_code")[
    ["under5_capacity", "total_capacity_computed"]
].sum()
current_supply.columns = ["current_under5_slots", "current_total_slots"]

population_df = population_df.join(current_supply, how="left")
population_df[["current_under5_slots", "current_total_slots"]] = population_df[
    ["current_under5_slots", "current_total_slots"]
].fillna(0)

# 3. Determine coverage requirements and identify child care deserts
# ------------------------------------------------------------------
population_df["required_total_slots"] = 0.33 * population_df["children_0_12"]
population_df.loc[population_df["high_demand"], "required_total_slots"] = (
    0.50 * population_df["children_0_12"]
)
population_df["required_under5_slots"] = 0.667 *␣
 ↪population_df["children_under5"]
population_df["required_total_slots"] = np.
 ↪ceil(population_df["required_total_slots"])
population_df["required_under5_slots"] = np.
 ↪ceil(population_df["required_under5_slots"])

desert_mask = (
    population_df["current_total_slots"] <=␣
 ↪population_df["required_total_slots"]
) | (population_df["current_under5_slots"] <=␣
 ↪population_df["required_under5_slots"])
desert_zips = population_df.index[desert_mask]

# 4. Filter potential new facility locations by distance constraints
# ------------------------------------------------------------------
location_df = location_df[location_df["zipcode"].isin(desert_zips)].copy()

existing_coords = childcare_df[["latitude", "longitude"]].dropna().values
if len(existing_coords) > 0:
    existing_tree = BallTree(np.deg2rad(existing_coords), metric="haversine")
    cand_coords = np.deg2rad(location_df[["latitude", "longitude"]].values)
    dist, _ = existing_tree.query(cand_coords, k=1)
    dist_miles = dist.ravel() * 3959.0
    too_close_mask = dist_miles < 0.06
    location_df = location_df.loc[~too_close_mask].copy()
```

```
selected_sites = []
for zipcode, group in location_df.groupby("zipcode"):
    coords = group[["latitude", "longitude"]].values
    n = len(coords)
    if n == 0:
        continue
    sorted_idx = np.argsort(coords[:, 0])
    lat = coords[sorted_idx, 0]
    lon = coords[sorted_idx, 1]
    removed = np.zeros(n, dtype=bool)
    for i_idx in range(n):
        i = sorted_idx[i_idx]
        if removed[i]:
            continue
        selected_sites.append(group.index[i])
        lat_i = lat[i_idx]
        lon_i = lon[i_idx]
        for j_idx in range(i_idx + 1, n):
            j = sorted_idx[j_idx]
            if removed[j]:
                continue
            if lat[j_idx] - lat_i > 0.0009:
                break

            avg_lat = math.radians((lat_i + lat[j_idx]) / 2.0)
            d_lat = (lat[j_idx] - lat_i) * 69.0
            d_lon = (lon[j_idx] - lon_i) * (69.0 * math.cos(avg_lat))
            dist = math.hypot(d_lat, d_lon)
            if dist < 0.06:
                removed[j] = True

location_df = location_df.loc[selected_sites].copy()


# 5. Build the optimization model
# -------------------------------
model = gp.Model("ChildCareDesertElimination")
model.Params.OutputFlag = 0

expansion_vars = {}
new_vars = {}

# 5.1. Expansion variables for existing facilities in desert areas
for idx, fac in childcare_df.iterrows():
    zip_code = fac["zip_code"]
    if zip_code not in desert_zips:
```

```
        continue
    nf = fac["total_capacity_computed"]
    if nf <= 0:
        continue

    exp_vars = {}
    exp_vars["x_total"] = model.addVar(
        vtype=GRB.INTEGER, lb=0, ub=math.floor(0.20 * nf)
    )
    exp_vars["x_u5"] = model.addVar(vtype=GRB.INTEGER, lb=0, ub=math.floor(0.20␣
 ↪* nf))
    exp_vars["y1"] = model.addVar(name=f"y1_{idx}", lb=0, ub=0.10 * nf)
    exp_vars["y2"] = model.addVar(name=f"y2_{idx}", lb=0, ub=0.05 * nf)
    exp_vars["y3"] = model.addVar(name=f"y3_{idx}", lb=0, ub=0.05 * nf)
    expansion_vars[idx] = exp_vars

    model.addConstr(
        exp_vars["x_total"] == exp_vars["y1"] + exp_vars["y2"] + exp_vars["y3"]
    )
    model.addConstr(exp_vars["x_u5"] <= exp_vars["x_total"])
    model.addConstr(exp_vars["y1"] <= 0.10 * nf)
    model.addConstr(exp_vars["y2"] <= 0.05 * nf)
    model.addConstr(exp_vars["y3"] <= 0.05 * nf)

# 5.2. New facility variables for candidate sites in desert areas
for idx, site in location_df.iterrows():
    zip_code = site["zipcode"]
    if zip_code not in desert_zips:
        continue
    var_small = model.addVar(vtype=GRB.BINARY, name=f"new_small_{idx}")
    var_med = model.addVar(vtype=GRB.BINARY, name=f"new_medium_{idx}")
    var_large = model.addVar(vtype=GRB.BINARY, name=f"new_large_{idx}")
    new_vars[idx] = {
        "small": var_small,
        "medium": var_med,
        "large": var_large,
        "zip": zip_code,
    }
    model.addConstr(var_small + var_med + var_large <= 1)

# 6. Coverage constraints for each desert ZIP code
# ------------------------------------------------
for zip_code in desert_zips:
    req_total = float(population_df.loc[zip_code, "required_total_slots"])
    req_under5 = float(population_df.loc[zip_code, "required_under5_slots"])
    curr_total = float(population_df.loc[zip_code, "current_total_slots"])
    curr_under5 = float(population_df.loc[zip_code, "current_under5_slots"])
```

```
    lhs_total = gp.LinExpr()
    lhs_under5 = gp.LinExpr()
    for f_idx, exp_vars in expansion_vars.items():
        if childcare_df.loc[f_idx, "zip_code"] != zip_code:
            continue
        lhs_total += exp_vars["x_total"]
        lhs_under5 += exp_vars["x_u5"]
    for i, vars_dict in new_vars.items():
        if vars_dict["zip"] != zip_code:
            continue
        lhs_total += (
            100 * vars_dict["small"]
            + 200 * vars_dict["medium"]
            + 400 * vars_dict["large"]
        )
        lhs_under5 += (
            50 * vars_dict["small"]
            + 100 * vars_dict["medium"]
            + 200 * vars_dict["large"]
        )

    additional_total_needed = max(0, math.ceil(req_total - curr_total) + 1)
    additional_under5_needed = max(0, math.ceil(req_under5 - curr_under5) + 1)
    model.addConstr(
        lhs_total >= additional_total_needed,␣
 ↪name=f"TotalCapacityReq_{zip_code}"
    )
    model.addConstr(
        lhs_under5 >= additional_under5_needed,␣
 ↪name=f"Under5CapacityReq_{zip_code}"
    )

# 7. Objective: Minimize total cost
# ---------------------------------
obj = gp.LinExpr()

# 7.1 Add expansion costs:
for f_idx, exp_vars in expansion_vars.items():
    nf = childcare_df.loc[f_idx, "total_capacity_computed"]
    if nf > 0:
        cost_per_slot_seg1 = (20000 + 200 * nf) / nf
        cost_per_slot_seg2 = (20000 + 400 * nf) / nf
        cost_per_slot_seg3 = (20000 + 1000 * nf) / nf
    else:
        cost_per_slot_seg1 = cost_per_slot_seg2 = cost_per_slot_seg3 = 0
    obj += cost_per_slot_seg1 * exp_vars["y1"]
```

```
    obj += cost_per_slot_seg2 * exp_vars["y2"]
    obj += cost_per_slot_seg3 * exp_vars["y3"]
    obj += 100.0 * exp_vars["x_u5"]

# 7.2 Add new facility construction costs:
for i, vars_dict in new_vars.items():
    obj += 70000.0 * vars_dict["small"]
    obj += 105000.0 * vars_dict["medium"]
    obj += 135000.0 * vars_dict["large"]

model.setObjective(obj, GRB.MINIMIZE)

# 8. Optimize the model
model.optimize()

# 9. Output the minimum total funding required
min_cost = model.ObjVal
min_cost = math.ceil(min_cost)
print(f"Min cost:${min_cost:,.2f}")
```

```
/var/folders/j5/g36rlrm17vv157kr1c8_ndyw0000gn/T/ipykernel_54991/306928679.py:41
: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  population_df["average income"].fillna(
/var/folders/j5/g36rlrm17vv157kr1c8_ndyw0000gn/T/ipykernel_54991/306928679.py:44
: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series
through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves as
a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  population_df["employment rate"].fillna(
```

```
Set parameter Username
```

```
Set parameter LicenseID to value 2722177
Academic license - for non-commercial use only - expires 2026-10-14
Min cost:$391,626,999.00
```

Fairness_Problem

October 28, 2025

0.1 Fairness + Budget

[20]:
```
# Objective: Maximize social coverage index under fairness constraint and $100M␣
↪budget

import pandas as pd
import numpy as np
import gurobipy as gp
from gurobipy import GRB, quicksum

# ---------------------------------------------------------------------
# 1. Load data
# ---------------------------------------------------------------------
POP_PATH = "./ChildCareDeserts_Data/population.csv"
INC_PATH = "./ChildCareDeserts_Data/avg_individual_income.csv"
EMP_PATH = "./ChildCareDeserts_Data/employment_rate.csv"
REG_PATH = "./ChildCareDeserts_Data/child_care_regulated.csv"

BUDGET = 100_000_000

FACILITY = {
    "S": (100, 65_000),
    "M": (200, 95_000),
    "L": (400, 115_000),
}  # (capacity, cost)


def norm_zip(df):
    zc = [c for c in df.columns if "zip" in c.lower()][0]
    df = df.rename(columns={zc: "ZIP"})
    df["ZIP"] = df["ZIP"].astype(str).str.zfill(5)
    return df


pop = norm_zip(pd.read_csv(POP_PATH))
inc = norm_zip(pd.read_csv(INC_PATH))
emp = norm_zip(pd.read_csv(EMP_PATH))
reg = norm_zip(pd.read_csv(REG_PATH))
```

```python
# Aggregate existing capacity by ZIP
cap_cols = [c for c in reg.columns if "capacity" in c.lower()]
reg["capacity"] = reg[cap_cols].sum(axis=1, numeric_only=True)
zip_cap = (
    reg.groupby("ZIP", as_index=False)["capacity"]
    .sum()
    .rename(columns={"capacity": "current_cap"})
)

# Simplify population
pop["pop_0_5"] = pop.filter(regex="0-5|under 5|0-5").sum(axis=1)
pop["pop_5_12"] = pop.filter(regex="5-9|10-14|5-9|10-14").sum(axis=1)
pop["pop_0_12"] = pop["pop_0_5"] + pop["pop_5_12"]

# Merge
zip_df = (
    pop.merge(zip_cap, on="ZIP", how="left")
    .merge(emp, on="ZIP", how="left")
    .merge(inc, on="ZIP", how="left")
    .fillna(0)
)

# High-demand flag
zip_df["high_demand"] = (zip_df.iloc[:, -2] >= 0.6) | (zip_df.iloc[:, -1] <=␣
↪60000)

# ------------------------------------------------------------------
# 2. Model
# ------------------------------------------------------------------
m = gp.Model("Fairness")

ZIPs = zip_df["ZIP"].tolist()

# Decision: number of facilities built
n_build = {
    (z, f): m.addVar(vtype=GRB.INTEGER, lb=0, name=f"build_{z}_{f}")
    for z in ZIPs
    for f in FACILITY
}

# Compute new capacities
new_cap = {z: quicksum(n_build[z, f] * FACILITY[f][0] for f in FACILITY) for z␣
↪in ZIPs}

# Coverage
cov05, cov012, covw = {}, {}, {}
```

```python
for i, row in zip_df.iterrows():
    z = row["ZIP"]
    pop05 = max(row["pop_0_5"], 1)
    pop012 = max(row["pop_0_12"], 1)
    cur = row["current_cap"]
    cov05[z] = (0.5 * cur + 0.5 * new_cap[z]) / pop05  # assume half slots for␣
↪0-5
    cov012[z] = (cur + new_cap[z]) / pop012
    covw[z] = (2 / 3) * cov05[z] + (1 / 3) * cov012[z]

# Fairness gap variables
min_cov = m.addVar(lb=0)
max_cov = m.addVar(lb=0)
for z in ZIPs:
    m.addConstr(covw[z] >= min_cov)
    m.addConstr(covw[z] <= max_cov)
m.addConstr(max_cov - min_cov <= 0.1)

# Desert elimination constraints
for i, row in zip_df.iterrows():
    z = row["ZIP"]
    if row["high_demand"]:
        m.addConstr(cov012[z] >= 0.5)
    else:
        m.addConstr(cov012[z] >= 1 / 3)
    m.addConstr(cov05[z] >= 2 / 3)

# Budget
total_cost = quicksum(n_build[z, f] * FACILITY[f][1] for z in ZIPs for f in␣
↪FACILITY)
m.addConstr(total_cost <= BUDGET)

# Objective: maximize weighted social coverage
social_index = quicksum((2 / 3) * cov05[z] + (1 / 3) * cov012[z] for z in ZIPs)
m.setObjective(social_index, GRB.MAXIMIZE)

# ---------------------------------------------------------------------
# 3. Solve
# ---------------------------------------------------------------------
m.Params.OutputFlag = 1
m.optimize()

# ---------------------------------------------------------------------
# 4. Results
# ---------------------------------------------------------------------
if m.Status == GRB.OPTIMAL:
    print(f"\nTotal cost used: {total_cost.getValue():,.0f}")
```

```
    print(f"Fairness gap: {(max_cov.X - min_cov.X):.3f}")
    print(
        f"Average weighted coverage: {np.mean([covw[z].getValue() for z in␣
↪ZIPs]):.3f}"
    )
else:
    print(
        "  Model infeasible - fairness or desert elimination may be too strict␣
↪under $100M."
    )
```

```
Set parameter OutputFlag to value 1
Gurobi Optimizer version 12.0.3 build v12.0.3rc0 (mac64[arm] - Darwin 24.6.0
24G231)

CPU model: Apple M3
Thread count: 8 physical cores, 8 logical processors, using up to 8 threads

Optimize a model with 6586 rows, 4940 columns and 27984 nonzeros
Model fingerprint: 0x632e8ec0
Variable types: 2 continuous, 4938 integer (0 binary)
Coefficient statistics:
  Matrix range     [4e-03, 1e+05]
  Objective range  [6e-03, 3e+02]
  Bounds range     [0e+00, 0e+00]
  RHS range        [1e-04, 1e+08]
Presolve removed 1057 rows and 0 columns
Presolve time: 0.00s

Explored 0 nodes (0 simplex iterations) in 0.01 seconds (0.01 work units)
Thread count was 1 (of 8 available processors)

Solution count 0

Model is infeasible or unbounded
Best objective -, best bound -, gap -
 Model infeasible - fairness or desert elimination may be too strict under
$100M.
```

0.2 Minimum budget to achieve fairness

[8]:
```
import math
import numpy as np
import pandas as pd
import gurobipy as gp
from gurobipy import GRB, quicksum
```

```python
POP_PATH = "./ChildCareDeserts_Data/population.csv"
INC_PATH = "./ChildCareDeserts_Data/avg_individual_income.csv"
EMP_PATH = "./ChildCareDeserts_Data/employment_rate.csv"
REG_PATH = "./ChildCareDeserts_Data/child_care_regulated.csv"

# New-build options (ZIP-level, no spatial constraints)
FACILITY_OPTIONS = [
    {"name": "S", "cap_total": 100, "cap_05_max": 50, "cost": 65000},
    {"name": "M", "cap_total": 200, "cap_05_max": 100, "cost": 95000},
    {"name": "L", "cap_total": 400, "cap_05_max": 200, "cost": 115000},
]


# ---------------------------
# Helpers
# ---------------------------
def normalize_zip(df):
    for c in df.columns:
        if "zip" in c.lower():
            df = df.rename(columns={c: "ZIP"})
            break
    df["ZIP"] = df["ZIP"].astype(str).str.zfill(5)
    return df


def safe_sum_cols(df, cols):
    return df[cols].sum(axis=1, numeric_only=True) if len(cols) > 0 else 0.0


def load_and_prepare():
    pop_df = pd.read_csv(POP_PATH)
    inc_df = pd.read_csv(INC_PATH)
    emp_df = pd.read_csv(EMP_PATH)
    reg_df = pd.read_csv(REG_PATH)

    pop_df = normalize_zip(pop_df)
    inc_df = normalize_zip(inc_df)
    emp_df = normalize_zip(emp_df)
    reg_df = normalize_zip(reg_df)

    # Population buckets → 0-12
    c_0_5 = [
        c
        for c in pop_df.columns
        if any(k in c.lower() for k in ["0-5", "0-5", "under 5"])
    ]
```

```
    c_5_9 = [c for c in pop_df.columns if any(k in c.lower() for k in ["5-9",␣
↪"5-9"])]
    c_10_14 = [
        c for c in pop_df.columns if any(k in c.lower() for k in ["10-14", "10-
14"])
    ]

    pop_df["pop_0_5"] = safe_sum_cols(pop_df, c_0_5)
    pop_df["pop_5_9"] = safe_sum_cols(pop_df, c_5_9)
    pop_df["pop_10_14"] = safe_sum_cols(pop_df, c_10_14)
    pop_df["pop_0_12"] = (
        pop_df["pop_0_5"] + pop_df["pop_5_9"] + 0.4 * pop_df["pop_10_14"]
    )

    inc_df = inc_df.rename(columns={"average income": "average_income"})
    emp_df = emp_df.rename(columns={"employment rate": "employment_rate"})

    # Facility capacities
    if "total_capacity" in reg_df.columns:
        reg_df["fac_capacity"] = reg_df["total_capacity"]
    else:
        cap_cols = [c for c in reg_df.columns if "capacity" in c.lower()]
        if len(cap_cols) == 0:
            raise ValueError("No capacity columns found in child_care_regulated.
↪csv")
        reg_df["fac_capacity"] = reg_df[cap_cols].sum(axis=1, numeric_only=True)

    u5_cols = [
        c
        for c in reg_df.columns
        if any(k in c.lower() for k in ["0-5", "0-5", "infant", "toddler",␣
↪"preschool"])
    ]
    reg_df["fac_u5_capacity"] = (
        reg_df[u5_cols].sum(axis=1, numeric_only=True) if len(u5_cols) > 0 else␣
↪0.0
    )

    # Per-ZIP current totals
    zip_total = (
        reg_df.groupby("ZIP", as_index=False)["fac_capacity"]
        .sum()
        .rename(columns={"fac_capacity": "current_total"})
    )
    zip_u5 = (
        reg_df.groupby("ZIP", as_index=False)["fac_u5_capacity"]
        .sum()
```

```
        .rename(columns={"fac_u5_capacity": "current_u5"})
    )

    zip_df = pd.DataFrame({"ZIP": sorted(set(pop_df["ZIP"]))})
    zip_df = (
        zip_df.merge(zip_total, on="ZIP", how="left")
        .merge(zip_u5, on="ZIP", how="left")
        .merge(pop_df[["ZIP", "pop_0_5", "pop_0_12"]], on="ZIP", how="left")
        .merge(emp_df[["ZIP", "employment_rate"]], on="ZIP", how="left")
        .merge(inc_df[["ZIP", "average_income"]], on="ZIP", how="left")
    )
    fill_cols = [
        "current_total",
        "current_u5",
        "pop_0_5",
        "pop_0_12",
        "employment_rate",
        "average_income",
    ]
    zip_df[fill_cols] = zip_df[fill_cols].fillna(0.0)

    return pop_df, inc_df, emp_df, reg_df, zip_df


def build_model(zip_df, reg_df, fairness_target=0.1, eps=1e-4):
    m = gp.Model("Eliminate_Deserts_Fairness")
    m.Params.OutputFlag = 0

    ZIPs = zip_df["ZIP"].tolist()

    # --- New builds per ZIP (S/M/L)
    n_build = {
        (z, opt["name"]): m.addVar(
            vtype=GRB.INTEGER, lb=0, name=f"build_{z}_{opt['name']}"
        )
        for z in ZIPs
        for opt in FACILITY_OPTIONS
    }
    new_total = {
        z: quicksum(
            n_build[(z, opt["name"])] * opt["cap_total"] for opt in␣
↪FACILITY_OPTIONS
        )
        for z in ZIPs
    }
    new_u5_cap = {
        z: quicksum(
```

```
            n_build[(z, opt["name"])] * opt["cap_05_max"] for opt in␣
↪FACILITY_OPTIONS
        )
        for z in ZIPs
    }
    new_u5 = {z: m.addVar(lb=0, vtype=GRB.CONTINUOUS, name=f"new_u5_{z}") for z␣
↪in ZIPs}
    for z in ZIPs:
        m.addConstr(new_u5[z] <= new_u5_cap[z], name=f"cap_u5_new_{z}")

    # --- Expansions per facility ( 20%, 3 segments)
    reg_df = (
        reg_df.reset_index(drop=True).reset_index().rename(columns={"index":␣
↪"f_id"})
    )
    F = [
        int(f)
        for f, cap in zip(reg_df["f_id"], reg_df["fac_capacity"])
        if (cap and cap > 0)
    ]
    fac_zip = {f: reg_df.loc[f, "ZIP"] for f in F}
    n_f = {f: float(reg_df.loc[f, "fac_capacity"]) for f in F}

    y1, y2, y3, x_total, x_u5 = {}, {}, {}, {}, {}
    for f in F:
        nf = n_f[f]
        y1[f] = m.addVar(lb=0, ub=0.10 * nf, vtype=GRB.CONTINUOUS,␣
↪name=f"y1_{f}")
        y2[f] = m.addVar(lb=0, ub=0.05 * nf, vtype=GRB.CONTINUOUS,␣
↪name=f"y2_{f}")
        y3[f] = m.addVar(lb=0, ub=0.05 * nf, vtype=GRB.CONTINUOUS,␣
↪name=f"y3_{f}")
        x_total[f] = m.addVar(lb=0, ub=0.20 * nf, vtype=GRB.CONTINUOUS,␣
↪name=f"x_{f}")
        x_u5[f] = m.addVar(lb=0, ub=0.20 * nf, vtype=GRB.CONTINUOUS,␣
↪name=f"x_u5_{f}")
        m.addConstr(x_total[f] == y1[f] + y2[f] + y3[f], name=f"sum_seg_{f}")
        m.addConstr(x_u5[f] <= x_total[f], name=f"u5_le_total_{f}")

    exp_total = {z: quicksum(x_total[f] for f in F if fac_zip[f] == z) for z in␣
↪ZIPs}
    exp_u5_zip = {z: quicksum(x_u5[f] for f in F if fac_zip[f] == z) for z in␣
↪ZIPs}

    # --- Coverage ratios per ZIP
    cov_tot, cov_u5, cov_w = {}, {}, {}
```

```
for _, row in zip_df.iterrows():
    z = row["ZIP"]
    curT = float(row["current_total"])
    curU = float(row["current_u5"])
    p12 = float(row["pop_0_12"])
    p05 = float(row["pop_0_5"])
    d12 = max(p12, 1.0)
    d05 = max(p05, 1.0)

    cov_tot[z] = (curT + exp_total[z] + new_total[z]) / d12
    cov_u5[z] = (curU + exp_u5_zip[z] + new_u5[z]) / d05
    cov_w[z] = (2.0 / 3.0) * cov_u5[z] + (1.0 / 3.0) * cov_tot[z]

# --- DESERT ERADICATION constraints (per prompt)
# High-demand if (employment_rate >= 60%) OR (average_income <= $60k)
emp = dict(zip(zip_df["ZIP"], zip_df["employment_rate"]))
inc = dict(zip(zip_df["ZIP"], zip_df["average_income"]))

for z in ZIPs:
    is_high = (emp[z] >= 0.60) or (inc[z] <= 60000.0)
    thr = 0.50 if is_high else (1.0 / 3.0)
    # Total coverage must strictly exceed threshold
    m.addConstr(cov_tot[z] >= thr + eps, name=f"desert_tot_{z}")
    # NYS policy: 0-5 coverage must be at least 2/3 of pop(0-5)
    m.addConstr(cov_u5[z] >= (2.0 / 3.0) + eps, name=f"u5_policy_{z}")

# --- Fairness band on weighted coverage (gap <= 0.1)
min_cov = m.addVar(lb=0, name="min_cov_w")
max_cov = m.addVar(lb=0, name="max_cov_w")
for z in ZIPs:
    m.addConstr(cov_w[z] >= min_cov, name=f"min_cov_{z}")
    m.addConstr(cov_w[z] <= max_cov, name=f"max_cov_{z}")
gap = m.addVar(lb=0, name="fairness_gap")
m.addConstr(gap == max_cov - min_cov, name="gap_def")
m.addConstr(gap <= float(fairness_target), name="gap_target")

# --- Costs (piecewise expansion + $100/slot for 0-5, builds from table)
exp_terms = []
for f in F:
    nf = n_f[f]
    c1 = (20000.0 + 200.0 * nf) / nf
    c2 = (20000.0 + 400.0 * nf) / nf
    c3 = (20000.0 + 1000.0 * nf) / nf
    exp_terms += [c1 * y1[f], c2 * y2[f], c3 * y3[f], 100.0 * x_u5[f]]

expansion_cost = quicksum(exp_terms)
build_cost = quicksum(
```

```
        n_build[(z, opt["name"])] * opt["cost"]
        for z in ZIPs
        for opt in FACILITY_OPTIONS
    )
    new_u5_cost = quicksum(100.0 * new_u5[z] for z in ZIPs)
    total_cost = expansion_cost + build_cost + new_u5_cost

    # --- Objective: minimize total cost (no budget)
    m.setObjective(total_cost, GRB.MINIMIZE)

    return m, {
        "ZIPs": ZIPs,
        "cov_tot": cov_tot,
        "cov_u5": cov_u5,
        "cov_w": cov_w,
        "gap": gap,
        "min_cov": min_cov,
        "max_cov": max_cov,
        "total_cost": total_cost,
    }


# ---------------------------
# Solve
# ---------------------------
pop_df, inc_df, emp_df, reg_df, zip_df = load_and_prepare()

m, R = build_model(zip_df, reg_df, fairness_target=0.1, eps=1e-4)
m.Params.OutputFlag = 1  # show logs
m.optimize()
```

```
Set parameter OutputFlag to value 1
Gurobi Optimizer version 12.0.3 build v12.0.3rc0 (mac64[arm] - Darwin 24.6.0
24G231)

CPU model: Apple M3
Thread count: 8 physical cores, 8 logical processors, using up to 8 threads

Optimize a model with 39438 rows, 84602 columns and 210220 nonzeros
Model fingerprint: 0xb2c6d293
Variable types: 79664 continuous, 4938 integer (0 binary)
Coefficient statistics:
  Matrix range     [2e-05, 4e+02]
  Objective range  [1e+02, 1e+05]
  Bounds range     [2e-01, 2e+02]
  RHS range        [1e-04, 3e+02]
Presolve removed 21444 rows and 49555 columns
Presolve time: 0.15s
```

```
Presolved: 17994 rows, 35047 columns, 89478 nonzeros
Variable types: 30475 continuous, 4572 integer (0 binary)

Root relaxation: objective 2.433663e+11, 9423 iterations, 0.04 seconds (0.05
work units)

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0 2.4337e+11    0 1523          - 2.4337e+11      -     -    0s
H    0     0                    2.434584e+11 2.4337e+11  0.04%     -    0s
H    0     0                    2.434255e+11 2.4337e+11  0.02%     -    0s
H    0     0                    2.434254e+11 2.4337e+11  0.02%     -    0s
     0     0 2.4339e+11    0 1577 2.4343e+11 2.4339e+11  0.01%     -    0s
H    0     0                    2.434216e+11 2.4339e+11  0.01%     -    0s
     0     0 2.4339e+11    0 1468 2.4342e+11 2.4339e+11  0.01%     -    0s
     0     0          -    0      2.4342e+11 2.4340e+11  0.01%     -    0s

Cutting planes:
  Gomory: 346
  MIR: 418
  StrongCG: 162

Explored 1 nodes (10188 simplex iterations) in 0.87 seconds (1.22 work units)
Thread count was 8 (of 8 available processors)

Solution count 4: 2.43422e+11 2.43425e+11 2.43426e+11 2.43458e+11

Optimal solution found (tolerance 1.00e-04)
Best objective 2.434215815767e+11, best bound 2.433974832668e+11, gap 0.0099%
```

[10]:
```python
# -------------------------------------------------------
# Extraction of results
# -------------------------------------------------------

if m.SolCount > 0:
    print("\n=== MODEL RESULTS ===")
    print(f"Total cost: ${m.objVal:,.2f}")
    print(f"Fairness gap: {R['gap'].X:.4f}\n")
```

```
=== MODEL RESULTS ===
Total cost: $243,421,581,576.70
Fairness gap: 0.1000
```